

CodroidApi.h函数接口说明

sendUserCommand

接口功能

发送用户命令，控制机器人状态

C++函数

```
Response sendUserCommand(UserCommand cmd, int timeout = 30)
```

C#函数

```
public async Task<Result> SetRobotMode(UserCommand command, int timeout = 30)
```

Python函数

```
def sendUserCommand(self, cmd: UserCommand, timeout: int = 30) -> Response:
```

参数说明

UserCommand: 机器人状态

SwitchOn	= 1,	// 上电
SwitchOff	= 2,	// 下电
ToReady	= 3,	// 进入手动模式
ToAuto	= 5,	// 进入自动模式
AcknowledgeError	= 100,	// 清除错误
ClearWarning	= 501,	// 清除警告
Recovery	= 1000,	// 恢复故障点模式

timeout: 超时等待时间，单位秒

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "id": 1,
  "type": "common",
  "action": "setparam",
  "data": [
    {
      "path": "Robot/Control/command",
      "value": 100
    }
  ]
}
```

返回参数

```
{
  "id": 1,
  "type": "common",
  "action": "setparam",
  "timestamp": 1709197381489,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    // 请求请错
    if 1
        res = api.sendUserCommand(c2::UserCommand::AcknowledgeError);
    printResponse(res);
    #endif
```

c#示例

```
res = await cobot.SetRobotMode(UserCommand.SwitchOff);
if (res.code == ResultCode.OK)
{
    Console.WriteLine("SetRobotMode is completed");
}
else
{
    Console.WriteLine("SetRobotMode is error: {0}", res.message);
}
```

python示例

```
res = api.sendUserCommand(UserCommand.ToReady)
printResponse(res)
```

getRobotState

接口功能

获取机器人状态

C++函数

```
Response getRobotState(int timeout = 5)
```

C#函数

```
public async Task<Result> GetRobotState(int timeout = 30)
```

python函数

```
def getRobotState(self, timeout: int = 30) -> Response:
```

参数说明

timeout: 超时等待时间，单位秒

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "id": 1,
  "type": "common",
  "action": "getparam",
  "data": [
    "Robot/Control/state"
  ]
}
```

返回参数

机器人当前状态 (Robot/Control/state)

- None = -1, // 未知
- Init = 0, // 初始化
- StandBy = 1, // 已下电
- Ready = 2, // 手动模式
- Rescue = 3, // 救援模式
- Auto = 4, // 自动模式
- Error = 6, // 机器人出错

```
{
  "id": 1,
  "type": "common",
  "action": "getparam",
  "timestamp": 1709195225410,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0,
    "data": {
      "Robot/Control/state": 4
    }
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    // 获取并打印机器人状态
    #if 1
        res = api.getRobotState();
        printResponse(res);
    #endif
}
```

c#示例

```
res = await cobot.GetRobotState();
if (res.code == ResultCode.OK)
{
    RobotState val = (RobotState)res.data!;
    Console.WriteLine("GetRobotState is completed,state: {0}", val);
    break;
    //if (val == RobotState.Ready)
    //{
    //    break;
    //}
}
else
{
    Console.WriteLine("GetRobotState is error: {0}", res.message);
    break;
}
```

python示例

```
res = api.getRobotState()
printResponse(res)
```

GetRobotStateFlag

接口功能

获取机器人状态标志

C++函数

```
Response GetRobotStateFlag(int timeout = 5)
```

C#函数

```
public async Task<Result> GetRobotStateFlag(int timeout = 30)
```

python函数

```
def getRobotStateFlag(self, timeout: int = 30) -> Response:
```

参数说明

timeout: 超时等待时间，单位秒

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

返回参数

```
返回值res.data包含
    data["robotMode"]
    data["safetyMode"]
    data["statusFlag"]
robotMode:
    "PowerOff"
    "Idle"
    "Jogging"
```

"Dragging"							
"ToPoint"							
"AutoReady"							
"AutoRunning"							
"Rescue"							
"Fault"							
"other"							
safetyMode	含义						
2	急停按下						
1	正常模式						
3	救援模式						
4	缩减模式						
0	存在错误						

statusFlag							
位0	位1	位2	位3	位4	位5	位6	位7
急停按下标志	上电标志	拖动中标志	机器人运动中标志	(未使用)	(未使用)	(未使用)	(未使用)
使用) (未使用)							
位8	位9	位10	位11	位12	位13	位14	位15
仿真标志	停止、运行、暂停	(未使用)	(未使用)	(未使用)	(未使用)	(未使用)	错误标志

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    #if 1
        res = api.getRobotStateFlag();
        printResponse(res);
    #endif
}
```

c#示例

```
res = await cobot.GetRobotStateFlag();
if (res.code == ResultCode.OK)
{
    RobotState val = res.data;
    Console.WriteLine("GetRobotState is completed,state: {0}", val);
}
else
{
    Console.WriteLine("GetRobotState is error: {0}", res.message);
    break;
}
```

python示例

```
res = api.getRobotStateFlag()
printResponse(res)
```

getPackPosition

接口功能

获取打包位

C++函数

```
Response getPackPosition(int timeout = 5)
```

python函数

```
def getPackPosition(self, timeout: int = 5) -> Response:
```

C#函数

```
public async Task<Result> GetPackPosition(int timeout = 5)
```

参数说明

timeout: 超时等待时间，单位秒

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "getparam",
  "data": [
    "Robot/Parameter/Mechanism/packingPosition"
  ],
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "id": 1,
  "type": "common",
  "action": "getparam",
  "timestamp": 1709263855386,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0,
    "data": {
      "Robot/Parameter/Mechanism/packingPosition": [
        89.0,
        0.0,
        148.3,
        -31.7,
        181.0,
        180.0
      ]
    }
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    // 获取打包位
    #if 1
        res = api.getPackPosition();
        printResponse(res);
    #endif
}
```

python示例

```
res = api.getPackPosition()
printResponse(res)
```

C#示例

goHome

接口功能

通过MovJonit运动到Home位.

C++函数

```
Response goHome(double speed = 60, double acc = 80, int timeout = 30)
```

C#函数

```
public Task<Result> GoHome(Speed speed, Acc acc, int timeout = 30)
```

python函数

```
def goHome(self, speed: float = 60, acc: float = 80, timeout: int = 30) ->
Response:
```

参数说明

speed: 运动速度，单位度/秒

acc: 运动加速度，单位度/秒平方

timeout: 指令发送超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "mov",
  "data": {
    "acc": {
      "aexjl": 0,
      "aexjr": 0,
      "aori": 0,
      "aper": 80.0,
      "atcp": 0
    },
    "speed": {
      "sexjl": 0,
      "sexjr": 0,
      "sori": 0,
      "sper": 60.0,
      "stcp": 0
    },
    "target": {
      "apos": {
        "jntpos1": 0.0,
        "jntpos2": 0.0,
        "jntpos3": 90.0,
        "jntpos4": 0.0,
        "jntpos5": 90.0,
        "jntpos6": 0.0
      }
    },
    "type": "movj"
  },
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "action": "mov",
  "code": 200,
  "data": {
    "code": 0,
    "msg": ""
  },
  "id": 1,
  "msg": "",
  "timestamp": 1709264295134,
  "type": "common"
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    // 通过MovJoint运动到Home位
    #if 1
        res = api.goHome();
        printResponse(res);
    #endif
}
```

c#示例

```
res = await cobot.GoHome(speed, acc);
if (res.code == ResultCode.OK)
{
    Console.WriteLine("GoHome is completed");
}
else
{
    Console.WriteLine("GoHome is error: {0}", res.message);
}
```

python示例

```
res = api.goHome()
printResponse(res)
```

setHomePosition

接口功能

设置Home位置

C++函数

```
void setHomePosition(double* position)
```

C#函数

```
public void SetHomePosition(double[] homePosition)
```

python函数

```
def setHomePosition(self, position: list) -> None:
```

参数说明

position double[6] 各关节旋转角度，单位度（deg）

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "mov",
  "data": {
    "acc": {
      "aexjl": 0,
      "aexjr": 0,
      "aori": 0,
      "aper": 80.0,
      "atcp": 0
    },
    "speed": {
      "sexjl": 0,
      "sexjr": 0,

```

```

        "sori": 0,
        "sper": 60.0,
        "stcp": 0
    },
    "target": {
        "apos": {
            "jntpos1": 0.0,
            "jntpos2": 0.0,
            "jntpos3": 90.0,
            "jntpos4": 0.0,
            "jntpos5": 90.0,
            "jntpos6": 0.0
        }
    },
    "type": "movj"
},
"id": 1,
"type": "common"
}

```

返回参数

```

{
    "id": 1,
    "type": "common",
    "action": "mov",
    "timestamp": 1709264746267,
    "code": 200,
    "msg": "",
    "data": {
        "msg": "",
        "code": 0
    }
}

```

cpp示例

```

#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    //设置Home点
    #if 1
        double homePosition[6] = { 0.0,0.0,90.0,0.0,90.0,0.0 };
        api.setHomePosition(homePosition);
        res = api.goHome();
        printResponse(res);
    #endif
}

```

```
#endif
```

c#函数示例

```
cobot.SetHomePosition(apos);
res = await cobot.GoHome(speed);
if (res.code == ResultCode.OK)
{
    Console.WriteLine("GoHome1 is completed");
}
else
{
    Console.WriteLine("GoHome1 is error: {0}", res.message);
}
```

python示例

goPack

接口功能

通过MovJonit运动到打包位

C++函数

```
Response goPack(double speed = 60, double acc = 80, int timeout = 30)
```

python函数

```
def goPack(self, speed: float = 60, acc: float = 80, timeout: int = 30) ->
Response:
```

C#函数

```
public Task<Result> GoPack(Speed speed, Acc acc, int timeout = 30)
```

参数说明

speed: 运动速度, 单位度/秒
acc: 运动加速度, 单位度/秒平方
timeout: 请求发送超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "mov",
  "data": {
    "acc": {
      "aexjl": 0,
      "aexjr": 0,
      "aori": 0,
      "aper": 80.0,
      "atcp": 0
    },
    "speed": {
      "sexjl": 0,
      "sexjr": 0,
      "sori": 0,
      "sper": 60.0,
      "stcp": 0
    },
    "target": {
      "apos": {
        "jntpos1": 89.0,
        "jntpos2": 0.0,
        "jntpos3": 148.3,
        "jntpos4": -31.7,
        "jntpos5": 181.0,
        "jntpos6": 180.0
      }
    },
    "type": "movj"
  },
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "action": "mov",
  "code": 200,
  "data": {
    "code": 0,

```



```
        "msg": ""
    },
    "id": 1,
    "msg": "",
    "timestamp": 1709265061402,
    "type": "common"
}
code: 0
msg: ok
data: {
    "code": 0,
    "msg": ""
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    //设置并回到pack点
    #if 1
        double packPosition[6] = { 0,0,150,-30,180,0 };
        api.setPackPosition(packPosition);
        res = api.goPack();
        printResponse(res);
    #endif
```

python示例

C#示例

movJ

接口功能

关节空间运动

C++函数

```
Response movJ(const Point& position, Speed speed, Acc acc, int timeout = 30)
```

C#函数

```
public async Task<Result> MovJoint(double[] position, Speed speed, Acc acc, int timeout = 30)
```

python函数

```
def movJ(self, position: Point, speed: Speed, acc: Acc, timeout: int = 30) -> Response:
```

参数说明

position: 目标位置, 类型为double[6], 数组分别为轴1~轴6的旋转角度, 单位度(deg), 例如[0,0,90,0,90,0]

speed:

joint = 0; // 关节速度, 适用关节空间运动, 单位: 度/秒(deg/s)

tcp = 0; // 末端线速度, 适用笛卡尔空间运动, 单位: 毫米/秒(mm/s)

ori = 0; // 末端旋转角速度, 适用笛卡尔空间运动, 单位: 度/秒(deg/s)

acc:

joint = 0; // 关节加速度, 适用关节空间运动, 单位: 度/秒平方(deg/s²)

tcp = 0; // 末端线速度, 适用笛卡尔空间运动, 单位: 毫米/秒平方(mm/s²)

ori = 0; // 末端旋转角速度, 适用笛卡尔空间运动, 单位: 度/秒平方(deg/s²)

timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "mov",
  "data": {
    "acc": {
      "aexj1": 0,
```

```

        "aexjr": 0,
        "aori": 0,
        "aper": 80.0,
        "atcp": 0
    },
    "speed": {
        "sexjl": 0,
        "sexjr": 0,
        "sori": 0,
        "sper": 60.0,
        "stcp": 0
    },
    "target": {
        "apos": {
            "jntpos1": 0.0,
            "jntpos2": 0.0,
            "jntpos3": 90.0,
            "jntpos4": 0.0,
            "jntpos5": 90.0,
            "jntpos6": 0.0
        }
    },
    "type": "movj"
},
"id": 1,
"type": "common"
}

```

返回参数

```

{
    "action": "mov",
    "code": 200,
    "data": {
        "code": 0,
        "msg": ""
    },
    "id": 1,
    "msg": "",
    "timestamp": 1709265061402,
    "type": "common"
}
code: 0
msg: ok
data: {
    "code": 0,
    "msg": ""
}

```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    //关节运动
    #if 1
        double apos[6] = { 0,0,90,0,90,0 };
        res = api.movJ(apos, speed, acc, 0);
        printResponse(res);
    #endif
```

c#示例

```
res = await cobot.MovJoint(apos, speed, acc);
if (res.code == ResultCode.OK)
{
    Console.WriteLine("MovJoint is completed");
}
else
{
    Console.WriteLine("MovJoint is error: {0}", res.message);
}
```

python示例

```
point1: Point = Point()
point1.type = PointType.Cart
point1.cpos.x = 455
point1.cpos.y = -114
point1.cpos.z = 372
point1.cpos.a = 172
point1.cpos.b = 1
point1.cpos.c = -91
res = api.movJ(point1)
print("----movJ----", end="\n")
printResponse(res)
```

movL

接口功能

直线运动

C++函数

```
Response movL(const Point& position, Speed speed = {0, 250, 80}, Acc acc = {0, 1200, 320}, int timeout = 30)
```

C#函数

```
public async Task<Result> MovLine(double[] position, Speed speed, Acc acc, int timeout = 30)
```

python函数

```
def movL(self, position: Point, speed: Speed = Speed(0, 250, 80), acc: Acc = Acc(0, 1200, 320), timeout: int = 30) -> Response:
```

参数说明

position: 目标位置, 类型为double[6], 表示[x,y,z,rx,ry,rz], (x,y,z)表示笛卡尔空间的位置, 单位毫米(mm), (rx,ry,rz)表示绕3个坐标轴的旋转角度, 单位度(deg)

speed:

joint = 0; // 关节速度, 适用关节空间运动, 单位: 度/秒(deg/s)

tcp = 0; // 末端线速度, 适用笛卡尔空间运动, 单位: 毫米/秒(mm/s)

ori = 0; // 末端旋转角速度, 适用笛卡尔空间运动, 单位: 度/秒(deg/s)

acc:

joint = 0; // 关节加速度, 适用关节空间运动, 单位: 度/秒平方(deg/s²)

tcp = 0; // 末端线速度, 适用笛卡尔空间运动, 单位: 毫米/秒平方(mm/s²)

ori = 0; // 末端旋转角速度, 适用笛卡尔空间运动, 单位: 度/秒平方(deg/s²)

timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "mov",
  "data": {
    "acc": {
      "aexjl": 0,
      "aexjr": 0,
      "aori": 320.0,
      "aper": 0,
      "atcp": 1200.0
    },
    "speed": {
      "sexjl": 0,
      "sexjr": 0,
      "sori": 40.0,
      "sper": 0,
      "stcp": 80.0
    },
    "target": {
      "cpos": {
        "a": 180.0,
        "b": 0.0,
        "c": -90.0,
        "poscfg": {
          "cf1": 0,
          "cf2": 0,
          "cf3": 0,
          "cf4": 0,
          "cf5": 0,
          "cf6": 0,
          "mode": -1
        },
        "x": 493.0,
        "y": 83.0,
        "z": 440.0
      },
      "type": "movl"
    },
    "id": 1,
    "type": "common"
  }
}
```

返回参数

```
{
  "action": "mov",
  "code": 200,
  "data": {
    "code": 0,
    "msg": ""
  },
  "id": 1,
  "msg": "",
  "timestamp": 1709265061402,
  "type": "common"
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    //直线移动
    #if 1
        c2::Point point1;
        point1.type = c2::PointType::Joint;
        point1.apos.jntPos[0] = 3;
        point1.apos.jntPos[1] = 12;
        point1.apos.jntPos[2] = 85;
        point1.apos.jntPos[3] = 0;
        point1.apos.jntPos[4] = 88;
        point1.apos.jntPos[5] = 4;
        res = api.movL(point1, speed, acc);
        std::cout << "----movL----" << std::endl;
        printResponse(res);
    #endif
```

c#示例

```
res = await cobot.MovLine(cpos, speed, acc);
if (res.code == ResultCode.OK)
{
    Console.WriteLine("MovLine is completed");
}
else
{
    Console.WriteLine("MovLine is error: {0}", res.message);
}
```

python示例

```
point2: Point = Point()
point2.type = PointType.Joint
point2.apos.jotPos[0] = 3
point2.apos.jotPos[1] = 12
point2.apos.jotPos[2] = 85
point2.apos.jotPos[3] = 0
point2.apos.jotPos[4] = 88
point2.apos.jotPos[5] = 4
res = api.movL(point2, speed)
print("----movL----", end="\n")
printResponse(res)
```

MovJointSegments

接口功能

多段关节空间运动

C++函数

```
Response movJointSegments(const MovJointSegments& segments, int timeout = 30)
```

C#函数

```
public async Task<Result> MovJointSegments(MovJointSegments segments, int timeout
= 30)
```

python函数

```
def movJointsSegments(self, segments: MovJointSegments, timeout: float = 30.0) ->
Response:
```

参数说明

segements

segements.AddmovJ(double[] position, Speed speed, Acc acc, ZoneType zoneType, Zone zone)

position: 目标位置，类型为double[6]，表示[x,y,z,rx,ry,rz]，(x,y,z)表示笛卡尔空间的位置，单位毫米（mm），（rx,ry,rz）表示绕3个坐标轴的旋转角度，单位度（deg）

speed: 运动速度，单位度/秒

acc: 运动加速度，单位度/秒平方

zoneType: 过渡类型


```
FINE, // 不过渡
RELATIVE, // 相对过渡
ABSOLUTE, // 绝对过渡
```

Zone:

per: 转弯百分比值，过渡类型为相对过渡时生效，适用所有运动类型
dis: 笛卡尔空间转弯区大小，，过渡类型为绝对过渡时生效，不适用MovJ

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

返回参数

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    #if 1

    #endif
```

python示例

c#示例

```
//Zone zone = new() {  
//    per = 50,  
//    dis = 10,  
//};  
  
//MovJointSegments movJointSegments = new ();  
//movJointSegments.AddMovJ(aPosS1, speed, acc, ZoneType.RELATIVE, zone);  
//movJointSegments.AddMovJ(aPosS2, speed, acc, ZoneType.RELATIVE, zone);  
//movJointSegments.AddMovJ(aPosS3, speed, acc, ZoneType.RELATIVE, zone);  
  
//res = await cobot.MovJointSegments(movJointSegments);  
//if (res.code == ResultCode.OK)  
//{  
//    Console.WriteLine("MovJointSegments is completed");  
//}  
//else  
//{  
//    Console.WriteLine("MovJointSegments is error: {0}", res.message);  
//}
```

MovCartSegments

接口功能

多段关节空间运动

C++函数

```
Response movCartSegments(const MovCartSegments& segments, int timeout = 30)
```

C#函数

```
public async Task<Result> MovCartSegments(MovCartSegments segments, int timeout = 30)
```

python函数

```
def movCartSegments(self, segments: MovCartSegments, timeout: int = 30) ->  
Response:
```

参数说明

segments 多段路径

segments.AddmovL(double[] position, Speed speed, Acc acc, ZoneType zoneType, Zone zone)

position: 目标位置，类型为double[6]，表示[x,y,z,rx,ry,rz]，(x,y,z)表示笛卡尔空间的位置，单位毫米（mm），（rx,ry,rz）表示绕3个坐标轴的旋转角度，单位度（deg）

speed: 运动速度，单位度/秒

acc: 运动加速度，单位度/秒平方

zoneType: 过渡类型

FINE, // 不过渡

RELATIVE, // 相对过渡

ABSOLUTE, // 绝对过渡

Zone:

per: 转弯百分比值，过渡类型为相对过渡时生效，适用所有运动类型

dis: 笛卡尔空间转弯区大小，，过渡类型为绝对过渡时生效，不适用MovJ

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

返回参数

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;
```

```

#if 1
    c2::Zone          zone{99, 200};
    c2::MovCartSegments segments;
    double            pp1[6] = {927.5, 214.5, 900, -180, 0, -90};
    double            pp2[6] = {1173.813, 51.721, 925.973, -168.473, 8.483,
-100.169};
    double            pp3[6] = {1077.021, 664.462, 659.728, 170.212, 20.967,
-66.149};
    segments.AddMovL(pp1, speed, acc, c2::ZoneType::Relative, zone);
    segments.AddMovL(pp2, speed, acc, c2::ZoneType::Relative, zone);
    segments.AddMovL(pp3, speed, acc, c2::ZoneType::Relative, zone);

    res = api.movCartSegments(segments);
    printResponse(res);
#endif

```

python示例

```

# zone: Zone = Zone(99, 200)
# segments: MovCartSegments = MovCartSegments()
# pp1: list[float] = [927.5, 214.5, 900, -180, 0, -90]
# pp2: list[float] = [1173.813, 51.721, 925.973, -168.473, 8.483, -100.169]
# pp3: list[float] = [1077.021, 664.462, 659.728, 170.212, 20.967, -66.149]
# pt1: Point = Point(PointType.Cart, cpos=Cpos(x=pp1[0], y=pp1[1], z=pp1[2],
a=pp1[3], b=pp1[4], c=pp1[5]))
# pt2: Point = Point(PointType.Cart, cpos=Cpos(x=pp2[0], y=pp2[1], z=pp2[2],
a=pp2[3], b=pp2[4], c=pp2[5]))
# pt3: Point = Point(PointType.Cart, cpos=Cpos(x=pp3[0], y=pp3[1], z=pp3[2],
a=pp3[3], b=pp3[4], c=pp3[5]))
# segments.AddMovL(pt1, speed, acc, ZoneType.Relative, zone)
# segments.AddMovL(pt2, speed, acc, ZoneType.Relative, zone)
# segments.AddMovL(pt3, speed, acc, ZoneType.Relative, zone)
#
# res = api.movCartSegments(segments)
# printResponse(res)

```

c#示例

```

Speed speed = new()
{
    joint = 90,
    tcp = 200,
    ori = 80
};

Acc acc = new()
{
    joint = 40,
    tcp = 1200,
    ori = 320
};

//double[] cpos1 = new double[6] {
//    564.243,
//    285.637,

```

```

//    316.408,
//    175.664,
//    6.551,
//    -102.436,
//};

//double[] cposS2 = new double[6] {
//    529.202,
//    43.006,
//    473.68,
//    -176.606,
//    13.134,
//    -100.731,
//};

//double[] cposS3 = new double[6] {
//    647.855,
//    76.044,
//    204.225,
//    -161.529,
//    -4.996,
//    -95.625,
//};

//Zone zone = new() {
//    per = 50,
//    dis = 10,
//};
//MovCartSegments movCartSegments = new();
//movCartSegments.AddMovL(cposS1, speed, acc, ZoneType.ABSOLUTE, zone);
//movCartSegments.AddMovL(cposS2, speed, acc, ZoneType.ABSOLUTE, zone);
//movCartSegments.AddMovL(cposS3, speed, acc, ZoneType.ABSOLUTE, zone);
//movCartSegments.AddMovC(cposS2,cposS1, speed,acc, ZoneType.RELATIVE, zone);

//res = await cobot.MovCartSegments(movCartSegments);
//if (res.code == ResultCode.OK)
//{
//    Console.WriteLine("MovCartSegments is completed");
//}
//else
//{
//    Console.WriteLine("MovCartSegments is error: {0}", res.message);
//}

```

movC

接口功能

圆弧运动

C++函数

```
Response movC(double* middlePosition,
              double* targetPosition,
              TcpSpeed speed = {250, 80},
              TcpAcc acc = {1200, 320},
              int timeout = 30)
```

python函数

```
def movC(self, middlePosition: Point, targetPosition: Point, speed: Speed =
Speed(0, 250, 80), acc: Acc = Acc(0, 1200, 320), timeout: int = 30) -> Response:
```

C#函数

```
public async Task<Result> MovC(Point middlePosition, Point targetPosition, Speed
speed, Acc acc, int timeout = 30)
```

参数说明

targetPosition: 目标位置，类型为double[6]，表示[x,y,z,rx,ry,rz]，(x,y,z)表示笛卡尔空间的位置，单位毫米（mm），（rx,ry,rz）表示绕3个坐标轴的旋转角度，单位度（deg）

middlePosition: 中间位置，数据类型同目标位置

speed: 运动速度，单位度/秒

acc: 运动加速度，单位度/秒平方

timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "mov",
  "data": {
    "acc": {
      "aexjl": 0,
      "aexjr": 0,
      "aori": 320.0,
```

```

        "aper": 0,
        "atcp": 1200.0
    },
    "middle": {
        "cpos": {
            "a": -179.999,
            "b": 0.001,
            "c": -90.0,
            "poscfg": {
                "cf1": 0,
                "cf2": 0,
                "cf3": 0,
                "cf4": 0,
                "cf5": 0,
                "cf6": 0,
                "mode": -1
            },
            "x": 493.928,
            "y": 226.189,
            "z": 391.195
        }
    },
    "speed": {
        "sexjl": 0,
        "sexjr": 0,
        "sori": 40.0,
        "sper": 0,
        "stcp": 80.0
    },
    "target": {
        "cpos": {
            "a": -179.999,
            "b": 0.002,
            "c": -89.999,
            "poscfg": {
                "cf1": 0,
                "cf2": 0,
                "cf3": 0,
                "cf4": 0,
                "cf5": 0,
                "cf6": 0,
                "mode": -1
            },
            "x": 514.573,
            "y": -140.393,
            "z": 481.725
        }
    },
    "type": "movc"
},
"id": 1,
"type": "common"
}

```

返回参数

```
{
  "action": "mov",
  "code": 200,
  "data": {
    "code": 0,
    "msg": ""
  },
  "id": 1,
  "msg": "",
  "timestamp": 1709265061402,
  "type": "common"
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    //圆弧运动
    #if 1
        double p_start[6] = {493.922,399.657,479.684,-180,0.001,-90};
        double p_cycle_middle[6] = { 493.928,226.189,391.195,-179.999,0.001,-90
        };
        double p_cycle_end[6] = {
            514.573,-140.393,481.725,-179.999,0.002,-89.999 };
        res = api.movL(p_start, speed);
        res = api.movC(p_cycle_middle, p_cycle_end);
        printResponse(res);
    #endif
}
```

python示例

```
point3: Point = Point()
point3.type = PointType.Joint
point3.apos.jotPos[0] = -9.9
point3.apos.jotPos[1] = 9.5
point3.apos.jotPos[2] = 87.5
point3.apos.jotPos[3] = -0.9
point3.apos.jotPos[4] = 90.2
point3.apos.jotPos[5] = -9

point4: Point = Point()
point4.type = PointType.Joint
point3.apos.jotPos[0] = 7.1
```



```
point3.apos.jotPos[1] = 3.1
point3.apos.jotPos[2] = 95
point3.apos.jotPos[3] = 0.4
point3.apos.jotPos[4] = 88
point3.apos.jotPos[5] = 8

res = api.movC(point3, point4, speed)
print("----movC----", end="\n")
printResponse(res)
```

stopMov

接口功能

让机器人停止运动

C++函数

```
Response stopMov(int timeout = 30)
```

C#函数

```
public async Task<Result> StopMov(int time = 250, int timeout = 3)
```

python函数

```
def stopMov(self, timeout: int = 30) -> Response:
```

参数说明

timeout: 请求发送超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "stopMov",
  "data": [],
  "id": 2,
  "type": "common"
}
```

返回参数

```
{
  "id": 2,
  "type": "common",
  "action": "stopMov",
  "timestamp": 1709272944569,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    //线程测试停止指令
    #if 1
        double pA[6] = { -13.11, -4.399, 94.22, -0.178, 90, -13.11 };
        double pB[6] = { 26, 50, 35, 2.9, 85, 10 };
        std::thread([&] {
            std::this_thread::sleep_for(std::chrono::seconds(1));
            auto res1 = api.stopMov();
            std::cout << "stop" << std::endl;
            printResponse(res1);
        }).detach();
        res = api.movJ(pA);
        printResponse(res);
        res = api.movJ(pB);
        printResponse(res);
    #endif
}
```

C#示例

```
Task task = Task.Run(async () =>
{
    Thread.Sleep(3000);
    Console.WriteLine("StopMov");
    res = await cobot.StopMov(200);
    if (res.code == ResultCode.OK)
    {
        Console.WriteLine("StopMov is completed");
    }
    else
    {
        Console.WriteLine("StopMov is error: {0}", res.message);
    }
});
```

python示例

jointJog

接口功能

关节点动. 该请求发送后, 如果保持点动持续, 需要持续调keepJog接口, 否则点动持续1s后会
自动停止

C++函数

```
Response jointJog(int jogIndex, JogSpeed jogSpeed, Direction direction, int  
timeout = 30)
```

C#函数

```
public async Task<Result> JointJog(int jogIndex, JogSpeed jogSpeed, Direction  
direction, int timeout = 30)
```

python函数

```
def jointJog(self, jogIndex: int, jogSpped: JogSpeed, direction: Direction,  
timeout: int = 30) -> Response:
```

参数说明

jogIndex: 需要点动的轴, 范围1~6
jogSpeed: 点动速度
 Low = 1, // 低速
 Mid = 2, // 中速
 High = 3, // 高速
direction: 点动方向
 Positive = 1, // 正方向
 Negative = -1, // 负方向
timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "setparam",
  "data": [
    {
      "path": "Robot/Control/jogMode",
      "value": 1
    },
    {
      "path": "Robot/Control/jogSpeed",
      "value": -1
    },
    {
      "path": "Robot/Control/jogIndex",
      "value": 1
    }
  ],
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "id": 1,
  "type": "common",
  "action": "setparam",
  "timestamp": 1709274175089,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    //关节持续点动
    #if 1
        for (int i = 1; i <= 6; i++) {
            c2::JogSpeed jogSpeed;
            c2::Direction direction;
            if (i % 3 == 1) {
                jogSpeed = c2::JogSpeed::Low;
            } else if (i % 3 == 2) {
                jogSpeed = c2::JogSpeed::Mid;
            } else {
                jogSpeed = c2::JogSpeed::High;
            }
            if (i % 2 == 0) {
                direction = c2::Direction::Positive;
            } else {
                direction = c2::Direction::Negative;
            }
            res = api.jointJog(i, jogSpeed, direction);
            printResponse(res);
            for (int j = 0; j < 3; j++) {
                std::this_thread::sleep_for(std::chrono::milliseconds(500));
                res = api.keepJog();
                printResponse(res);
            }
        }
        res = api.stopJog();
        printResponse(res);
    }
```

```
#endif
```

C#示例

```
res = await cobot.JointJog(i, jogSpeed, direction);
if (res.code == ResultCode.OK)
{
    Console.WriteLine("JointJog is completed");
}
else
{
    Console.WriteLine("JointJog is error: {0}", res.message);
}
```

python示例

```
for i in range(1, 7):
    jogSpeed: JogSpeed = JogSpeed.Low
    direction: Direction = Direction.Negative

    if i % 3 == 1:
        jogSpeed = JogSpeed.Low
    elif i % 3 == 2:
        jogSpeed = JogSpeed.Mid
    else:
        jogSpeed = JogSpeed.High

    if i % 2 == 0:
        direction = Direction.Positive
    else:
        direction = Direction.Negative

    res = api.jointJog(i, jogSpeed, direction)
    printResponse(res)

    for j in range(3):
        time.sleep(0.5)
        res = api.keepJog()
        printResponse(res)

res = api.stopJog()
printResponse(res)
```

tcpJog

接口功能

末端点动. 该请求发送后, 如果要保持点动持续, 需要持续调keepJog接口, 否则点动持续1s后会
自动停止

C++函数

```
Response tcpJog(int jogIndex, JogSpeed jogSpeed, Direction direction, int timeout = 30)
```

C#函数

```
public async Task<Result> TcpJog(int jogIndex, JogSpeed jogSpeed, Direction direction, int timeout = 30)
```

python函数

```
def tcpJog(self, jogIndex: int, jogSpeed: JogSpeed, direction: Direction, timeout: int = 30) -> Response:
```

参数说明

jogIndex: 1~6 分别对应 x,y,z,rx,ry,rz
jogSpeed: 点动速度
direction: 点动方向
timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "setparam",
  "data": [
    {
      "path": "Robot/Control/jogMode",
      "value": 2
    },
    {
      "path": "Robot/Control/jogSpeed",
      "value": -1
    },
    {
      "path": "Robot/Control/jogIndex",
```

```
        "value": 1
    }
],
"id": 1,
"type": "common"
}
```

返回参数

```
{
  "id": 1,
  "type": "common",
  "action": "setparam",
  "timestamp": 1709274688120,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    //末端持续点动
    #if 1
        for (int i = 1; i <= 6; i++) {
            c2::JogSpeed jogSpeed;
            c2::Direction direction;
            if (i % 3 == 1) {
                jogSpeed = c2::JogSpeed::Low;
            } else if (i % 3 == 2) {
                jogSpeed = c2::JogSpeed::Mid;
            } else {
                jogSpeed = c2::JogSpeed::High;
            }
            if (i % 2 == 0) {
                direction = c2::Direction::Positive;
            } else {
                direction = c2::Direction::Negative;
            }
            res = api.tcpJog(i, jogSpeed, direction);
            printResponse(res);
            for (int j = 0; j < 3; j++) {
```



```

        std::this_thread::sleep_for(std::chrono::milliseconds(500));
        res = api.keepJog();
        printResponse(res);
    }
}
res = api.stopJog();
printResponse(res);
#endif

```

C#示例

```

res = await cobot.TcpJog(i, jogSpeed, direction);
if (res.code == ResultCode.OK)
{
    Console.WriteLine("JointJog is completed");
}
else
{
    Console.WriteLine("JointJog is error: {0}", res.message);
}

```

python示例

keepJog

接口功能

保持当前点动. 以500毫秒左右的频率持续调该接口，可保持点动持续

C++函数

```
Response keepJog(int timeout = 30)
```

C#函数

```
public async Task<Result> KeepJog(int timeout = 30)
```

python函数

```
def keepJog(self, timeout: int = 30) -> Response:
```

参数说明

timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "setparam",
  "data": [
    {
      "path": "Robot/Control/commandHeart",
      "value": 1709274626825
    }
  ],
  "id": 2,
  "type": "common"
}
```

返回参数

```
{
  "id": 2,
  "type": "common",
  "action": "setparam",
  "timestamp": 1709274958018,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
```

```

SetConsoleOutputCP(CP_UTF8);

c2::CodroidApi api("192.168.101.100", "9000");
c2::Response res;

//末端持续点动
#if 1
    for (int i = 1; i <= 6; i++) {
        c2::JogSpeed jogSpeed;
        c2::Direction direction;
        if (i % 3 == 1) {
            jogSpeed = c2::JogSpeed::Low;
        } else if (i % 3 == 2) {
            jogSpeed = c2::JogSpeed::Mid;
        } else {
            jogSpeed = c2::JogSpeed::High;
        }
        if (i % 2 == 0) {
            direction = c2::Direction::Positive;
        } else {
            direction = c2::Direction::Negative;
        }
        res = api.tcpJog(i, jogSpeed, direction);
        printResponse(res);
        for (int j = 0; j < 3; j++) {
            std::this_thread::sleep_for(std::chrono::milliseconds(500));
            res = api.keepJog();
            printResponse(res);
        }
    }
    res = api.stopJog();
    printResponse(res);
#endif

```

C#示例

```

res = await cobot.TcpJog(i, jogSpeed, direction);
if (res.code == ResultCode.OK)
{
    Console.WriteLine("JointJog is completed");
}
else
{
    Console.WriteLine("JointJog is error: {0}", res.message);
}

for (int j = 0; j < 3; j++)
{
    Thread.Sleep(500);
    res = await cobot.KeepJog();
    if (res.code == ResultCode.OK)
    {
        Console.WriteLine("KeepJog is completed");
    }
    else

```

```
{  
    Console.WriteLine("KeepJog is error: {0}", res.message);  
}  
}  
}
```

python示例

```
for i in range(1, 7):  
    jogSpeed: JogSpeed = JogSpeed.Low  
    direction: Direction = Direction.Negative  
  
    if i % 3 == 1:  
        jogSpeed = JogSpeed.Low  
    elif i % 3 == 2:  
        jogSpeed = JogSpeed.Mid  
    else:  
        jogSpeed = JogSpeed.High  
  
    if i % 2 == 0:  
        direction = Direction.Positive  
    else:  
        direction = Direction.Negative  
  
    res = api.jointJog(i, jogSpeed, direction)  
    printResponse(res)  
  
    for j in range(3):  
        time.sleep(0.5)  
        res = api.keepJog()  
        printResponse(res)  
  
res = api.stopJog()  
printResponse(res)
```

stopJog

接口功能

停止点动

C++函数

```
Response stopJog(int timeout = 30)
```

C#函数

```
public async Task<Result> StopJog(int timeout = 30)
```

python函数

```
def stopJog(self, timeout: int = 30) -> Response:
```

参数说明

timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "setparam",
  "data": [
    {
      "path": "Robot/Control/jogMode",
      "value": 0
    },
    {
      "path": "Robot/Control/jogSpeed",
      "value": 0
    },
    {
      "path": "Robot/Control/jogIndex",
      "value": 0
    }
  ],
  "id": 25,
  "type": "common"
}
```

返回参数

```
{
  "id": 25,
  "type": "common",
  "action": "setparam",
  "timestamp": 1709274958018,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    //末端持续点动
    #if 1
        for (int i = 1; i <= 6; i++) {
            c2::JogSpeed jogSpeed;
            c2::Direction direction;
            if (i % 3 == 1) {
                jogSpeed = c2::JogSpeed::Low;
            } else if (i % 3 == 2) {
                jogSpeed = c2::JogSpeed::Mid;
            } else {
                jogSpeed = c2::JogSpeed::High;
            }
            if (i % 2 == 0) {
                direction = c2::Direction::Positive;
            } else {
                direction = c2::Direction::Negative;
            }
            res = api.tcpJog(i, jogSpeed, direction);
            printResponse(res);
            for (int j = 0; j < 3; j++) {
                std::this_thread::sleep_for(std::chrono::milliseconds(500));
                res = api.keepJog();
                printResponse(res);
            }
        }
        res = api.stopJog();
        printResponse(res);
    }
```

```
#endif
```

C#示例

```
res = await cobot.StopJog();
if (res.code == ResultCode.OK)
{
    Console.WriteLine("StopJog is completed");
}
else
{
    Console.WriteLine("StopJog is error: {0}", res.message);
}
```

python示例

```
for i in range(1, 7):
    jogSpeed: JogSpeed = JogSpeed.Low
    direction: Direction = Direction.Negative

    if i % 3 == 1:
        jogSpeed = JogSpeed.Low
    elif i % 3 == 2:
        jogSpeed = JogSpeed.Mid
    else:
        jogSpeed = JogSpeed.High

    if i % 2 == 0:
        direction = Direction.Positive
    else:
        direction = Direction.Negative

    res = api.jointJog(i, jogSpeed, direction)
    printResponse(res)

    for j in range(3):
        time.sleep(0.5)
        res = api.keepJog()
        printResponse(res)

res = api.stopJog()
printResponse(res)
```

getJointPosition

接口功能

获取当前关节位置

C++函数

```
Response getJointPosition(int timeout = 30)
```

C#函数

```
public async Task<Result> GetJointPosition(int timeout = 30)
```

python函数

```
def getJointPosition(self, timeout: int = 30) -> Response:
```

参数说明

timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "getCurAPos",
  "data": [],
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "action": "getCurAPos",
  "code": 200,
  "data": {
    "code": 0,
    "data": {
      "exjntpos1": 0.0,
      "exjntpos10": 0.0,
      "exjntpos2": 0.0,
```



```

        "exjntpos3": 0.0,
        "exjntpos4": 0.0,
        "exjntpos5": 0.0,
        "exjntpos6": 0.0,
        "exjntpos7": 0.0,
        "exjntpos8": 0.0,
        "exjntpos9": 0.0,
        "jntpos1": 3.689346313476563,
        "jntpos2": -3.01025390625,
        "jntpos3": 114.17472839355469,
        "jntpos4": 32.76741027832031,
        "jntpos5": 80.53871154785156,
        "jntpos6": -13.923797607421875,
        "jntpos7": 0.0
    },
    "msg": ""
},
"id": 1,
"msg": "",
"timestamp": 1709276636090,
"type": "common"
}

```

cpp示例

```

#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    //获取当前关节位置
    #if 1
        res = api.getJointPosition();
        printResponse(res);
        std::vector<double> p14 = res.data;
        std::cout << "vector p14: [";
        for (auto& d : p14) {
            std::cout << d << ", ";
        }
        std::cout << "]" << std::endl;
        double p15[6] = {res.data[0], res.data[1], res.data[2], res.data[3],
            res.data[4], res.data[5]};
        std::cout << "double p15[6]: [" << p15[0] << ", " << p15[1] << ", " <<
            p15[2] << ", " << p15[3] << ", " << p15[4] << ", "
            << p15[5] << "]" << std::endl;
    #endif
}

```

C#示例

python示例

```
res = api.getJointPosition()  
printResponse(res)
```

getCartPosition

接口功能

获取当前笛卡尔坐标位置

C++函数

```
Response getCartPosition(int timeout = 30)
```

C#函数

```
public async Task<Result> GetCartPosition(int timeout = 30)
```

python函数

```
def getCartPosition(self, timeout: int = 30) -> Response:
```

参数说明

timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "getCurCPos",
  "data": [],
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "action": "getCurCPos",
  "code": 200,
  "data": {
    "code": 0,
    "data": {
      "x": 463.5961393047536,
      "y": 250.13618473916466,
      "z": 359.05392239225284,
      "a": -166.3883930147858,
      "b": 6.193869247124272,
      "c": -70.68347995655591,
      "e": 0.0,
      "exjntpos1": 0.0,
      "exjntpos2": 0.0,
      "exjntpos3": 0.0,
      "exjntpos4": 0.0,
      "exjntpos5": 0.0,
      "exjntpos6": 0.0,
      "exjntpos7": 0.0,
      "exjntpos8": 0.0,
      "exjntpos9": 0.0,
      "exjntpos10": 0.0,
      "poscfg": {
        "mode": 0,
        "cf1": 0,
        "cf2": 0,
        "cf3": 0,
        "cf4": 0,
        "cf5": 0,
        "cf6": 0,
        "cf7": 0
      }
    },
    "msg": ""
  },
  "id": 1,
  "msg": "",
  "timestamp": 1709276636090,
  "type": "common"
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    //获取当前笛卡尔位姿
    #if 1
        res = api.getCartPosition();
        printResponse(res);
        std::vector<double> p16 = res.data;
        std::cout << "vector p16: [";
        for (auto& d : p16) {
            std::cout << d << ", ";
        }
        std::cout << "]" << std::endl;
        double p17[6] = {res.data[0], res.data[1], res.data[2], res.data[3],
            res.data[4], res.data[5]};
        std::cout << "double p17[6]: [" << p17[0] << ", " << p17[1] << ", " <<
            p17[2] << ", " << p17[3] << ", " << p17[4]
            << ", " << p17[5] << "]" << std::endl;
    #endif
}
```

C#示例

python示例

```
res = api.getCartPosition()
printResponse(res)
```

setDO

接口功能

设置数字输出端口值

C++函数

```
Response setDO(int port, int val, int timeout = 30)
```

C#函数

```
public async Task<Result> SetDO(int port, int val, int timeout = 30)
```

python函数

```
def setDO(self, port: int, val: int, timeout: int = 30) -> Response:
```

参数说明

port: 数字输出端口号
val: 数字输出值, 只能是0或1
timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{  
  "action": "setDO",  
  "data": {  
    "port": 1,  
    "val": 0  
  },  
  "id": 1,  
  "type": "common"  
}
```

返回参数

```
{
  "action": "setDO",
  "code": 200,
  "data": {
    "code": 0,
    "msg": ""
  },
  "id": 1,
  "msg": "",
  "timestamp": 1709278265963,
  "type": "common"
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    //设置输出
    #if 1
        res = api.setDO(1,0);
        printResponse(res);
    #endif
}
```

C#示例

```
res = await cobot.SetDO(4, 1);
if (res.code == ResultCode.OK)
{
    Console.WriteLine("SetDO is completed");
}
else
{
    Console.WriteLine("SetDO is error: {0}", res.message);
}
```

python示例

```
res = api.setDO(1, 0)
printResponse(res)
```

getDI

接口功能

获取数字输入端口值

C++函数

```
Response getDI(int port, int timeout = 30)
```

C#函数

```
public async Task<Result> GetDI(int port, int timeout = 30)
```

python函数

```
def getDI(self, port: int, timeout: int = 30) -> Response:
```

参数说明

port: 需要获取值的端口号
timeout: 发送请求超时时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "getDI",
  "data": {
    "port": 0
  },
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "action": "getDI",
  "code": 200,
  "data": {
    "code": 0,
    "data": 0,
    "msg": ""
  },
  "id": 1,
  "msg": "",
  "timestamp": 1709279833380,
  "type": "common"
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    //获取输入
    #if 1
        res = api.getDI(0);
        printResponse(res);
    #endif
}
```

C#示例

```
res = await cobot.GetDI(1);
if (res.code == ResultCode.OK)
{
    int val = (int)res.data!;
    Console.WriteLine("GetDI is completed,val: {0}", val);
}
else
{
    Console.WriteLine("GetDI is error: {0}", res.message);
}
```


python示例

```
res = api.getDI(0)
printResponse(res)
```

setDOGroup

接口功能

设置连续多个数字输出端口值

C++函数

```
Response setDOGroup(int startPort, int endPort, int val, int timeout = 30)
```

C#函数

```
public async Task<Result> SetDOGroup(int startPort, int endPort, int val, int
timeout = 30)
```

python函数

```
def setDOGroup(self, startPort: int, endPoint: int, val: int, timeout: int = 30)
-> Response:
```

参数说明

startPort: 开始端口，二进制低位
endPort: 结束端口，二进制高位
val: 设定二进制值（例如**startPort = 0, endPort = 2, val=5**，代表从端口0到端口2为101）

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "setDOGroup",
  "data": {
    "endPort": 2,
    "startPort": 0,
    "val": 1
  },
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "action": "setDOGroup",
  "code": 200,
  "data": {
    "code": 0,
    "msg": ""
  },
  "id": 1,
  "msg": "",
  "timestamp": 1709280188636,
  "type": "common"
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    //获取输入
    if 1
        res = api.setDOGroup(0, 2, 1);
        printResponse(res);
    endif
```

C#示例

```
res = await cobot.SetDOGroup(0, 4, 11);
if (res.code == ResultCode.OK)
{
    Console.WriteLine("SetDO is completed");
}
else
{
    Console.WriteLine("SetDO is error: {0}", res.message);
}
```

python示例

```
res = api.setDOGroup(0, 2, 0)
printResponse(res)
```

getDIGroup

接口功能

获取连续数字输入端口值将其作为二进制数

C++函数

```
Response getDIGroup(int startPort, int endPort, int timeout = 30)
```

C#函数

```
public async Task<Result> GetDIGroup(int startPort, int endPort, int timeout = 30)
```

python函数

```
def getDIGroup(self, startPort: int, endPoint: int, timeout: int = 30) ->
Response:
```

参数说明

startPort: 开始端口，读取作为二进制低位
endPort: 结束端口，读取作为二进制高位
timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "getDIGroup",
  "data": {
    "endPort": 3,
    "startPort": 0
  },
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "id": 1,
  "type": "common",
  "action": "getDIGroup",
  "timestamp": 1709280418136,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0,
    "data": 0
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    //获取输入
```

```
#if 1
    res = api.getDIGroup(0, 3);
    printResponse(res);
#endif
```

C#函数

```
res = await cobot.GetDIGroup(2, 3);
if (res.code == ResultCode.OK)
{
    int val = (int)res.data!;
    Console.WriteLine("GetDI is completed,val: {0}", val);
}
else
{
    Console.WriteLine("GetDI is error: {0}", res.message);
}
```

python函数

```
res = api.getDIGroup(0, 3)
printResponse(res)
```

setCurrentCoor

接口功能

设置当前坐标系.

C++函数

```
Response setCurrentCoor(const std::string& varName, UserCoor coor, int timeout = 30)
```

python函数

```
def setCurrentCoor(self, vaName: str, coor: UserCoor, timeout: int = 30) -> Response:
```

C#函数

```
public async Task<Result> SetCurrentCoor(string varName, UserCoor coor, int timeout = 30)
```

参数说明

`varName` 坐标系变量名
`coord` 坐标系参数
`timeout`: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "setCurrentCoord",
  "data": {
    "key": "varName",
    "value": {
      "USERCOORD": {
        "x": x,
        "y": y,
        "z": z,
        "a": a,
        "b": b,
        "c": c
      }
    }
  },
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "id": 1,
  "type": "common",
  "action": "setCurrentCoor",
  "timestamp": 1709280418136,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0,
    "data": 0
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    // 设置当前坐标系
    #if 1
        c2::UserCoor coor;
        coor.x = 110.2;
        coor.y = 111.2;
        coor.z = 112.2;
        coor.a = 113.2;
        coor.b = 114.2;
        coor.c = 115.2;

        res = api.setCurrentCoor("coor1", coor);
        printResponse(res);
    #endif
```

python示例

```
coor: UserCoor = UserCoor()
coor.x = 110.2
coor.y = 111.2
coor.z = 112.2
coor.a = 113.2
coor.b = 114.2
coor.c = 115.2

res = api.setCurrentCoor("coor1", coor)
printResponse(res)
```

C#示例

setCurrentTool

接口功能

设置当前工具.

C++函数

```
Response setCurrentTool(const std::string& varName, Tool tool, int timeout = 30)
```

python函数

```
def setCurrentTool(self, varName: str, tool: Tool, timeout: int = 30) ->
Response:
```

C#函数

```
public async Task<Result> SetCurrentTool(string varName, Tool tool, int timeout =
30)
```

参数说明

varName 工具变量名
tool 工具参数
timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "setCurrentCoor",
  "data": {
    "key": "varName",
    "value": {
      "TOOL": {
        "x": x,
        "y": y,
        "z": z,
        "a": a,
        "b": b,
        "c": c,
        "dyn": {
          "m": m
        },
        "pos": {
          "mx": mx,
          "my": my,
          "mz": mz
        },
        "tensor": {
          "ixx": ixx,
          "ixy": ixy,
          "ixz": ixz,
          "iyy": iyy,
          "iyz": iyz,
          "izz": izz
        }
      }
    }
  },
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "id": 1,
  "type": "common",
  "action": "setCurrentTool",
  "timestamp": 1709280418136,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0,
    "data": 0
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    // 设置当前工具
    #if 0
        c2::Tool tool;
        tool.x = 110.2;
        tool.y = 111.2;
        tool.z = 112.2;
        tool.a = 113.2;
        tool.b = 114.2;
        tool.c = 115.2;

        c2::LoadDyn& dyn = tool.dyn;
        dyn.M = 2;
        dyn.pos.Mx = 5.3;
        dyn.pos.My = 10.1;
        dyn.pos.Mz = 15.2;

        res = api.setCurrentTool("tool1", tool);
        printResponse(res);
    #endif
```

python示例

```
tool: Tool = Tool()
tool.x = 110.2
tool.y = 111.2
tool.z = 112.2
tool.a = 113.2
tool.b = 114.2
tool.c = 115.2

dyn: LoadDyn = tool.dyn
dyn.mass = 2
dyn.centerPos.Mx = 5.3
dyn.centerPos.My = 10.1
dyn.centerPos.Mz = 15.2

res = api.setCurrentTool("tool1", tool)
printResponse(res)
```

setCurrentPayload

接口功能

设置当前负载

C++函数

```
Response setCurrentPayload(const std::string& varName, LoadDyn dyn, int timeout = 30)
```

Python

```
def setCurentPayload(self, varName: str, dyn: LoadDyn, timeout: int = 30) -> Response:
```

C#函数

```
public async Task<Result> SetCurrentPayload(string varName, LoadDyn dyn, int timeout = 30)
```

参数说明

varName 负载变量名
dyn 负载参数
timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "setCurrentCoor",
  "data": {
    "key": "varName",
    "value": {
      "PAYLOAD": {
        "dyn": {
          "m": m
        },
        "pos": {
          "mx": mx,
          "my": my,
          "mz": mz
        },
        "tensor": {
          "ixx": ixx,
          "ixy": ixy,
          "ixz": ixz,
          "iyy": iyy,
          "iyz": iyz,
          "izz": izz
        }
      }
    }
  },
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "id": 1,
  "type": "common",
  "action": "setcurpayload",
  "timestamp": 1709280418136,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0,
    "data": 0
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    // 设置当前工具
    #if 1
        c2::LoadDyn dyn;
        dyn.M = 2;
        dyn.pos.Mx = 5.3;
        dyn.pos.My = 10.1;
        dyn.pos.Mz = 15.2;

        res = api.setCurrentPayload("payload1", dyn);
        printResponse(res);
    #endif
```

getProjectState

接口功能

获取工程状态

C++函数

```
Response getProjectState(int timeout = 30)
```

C#函数

```
public async Task<Result> GetProjectState(int timeout = 30)
```

python函数

```
def getProjectState(self, timeout: int = 30) -> Response:
```

参数说明

projectName 工程名（避免使用中文字符）
taskName 程序名（避免使用中文字符）
label 标签名（避免使用中文字符）
timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "getProjectState",
  "data": {
  },
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "id": 1,
  "type": "common",
  "action": "getProjectState",
  "timestamp": 1709280418136,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0,
    "data": 0
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    // 获取工程状态
    #if 1
        res = api.getProjectState();
        printResponse(res);
    #endif
```

C#示例

python示例

```
res = api.getProjectState()
printResponse(res)
```

runProject

接口功能

运行指定工程

C++函数

```
Response runProject(const std::string& projectName,  
                    const std::string& taskName = "",  
                    const std::string& label    = "",  
                    int                 timeout  = 30)
```

C#函数

```
public async Task<Result> RunProject(string projectName, string taskName = "",  
string label = "", int timeout = 30)
```

python函数

```
def runProject(self, projectName: str, taskName: str, label: str, timeout: int =  
30) -> Response:
```

参数说明

projectName 工程名（避免使用中文字符）
taskName 程序名（避免使用中文字符）
label 标签名（避免使用中文字符）
timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "getProjectState",
  "data": {
    "projectName": projectName,
    "taskName": taskName,
    "taskName": label
  },
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "id": 1,
  "type": "common",
  "action": "runProject",
  "timestamp": 1709280418136,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0,
    "data": 0
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    // 工程运行控制接口测试
    #if 0

        res = api.runProject("test1");
        printResponse(res);

        std::this_thread::sleep_for(std::chrono::seconds(2));

        res = api.stopProject();
        printResponse(res);

        std::this_thread::sleep_for(std::chrono::seconds(2));
```

```

    res = api.runProject("test2");
    printResponse(res);

    std::this_thread::sleep_for(std::chrono::seconds(2));

    res = api.stopProject();
    printResponse(res);

    res = api.getProjectState();
    printResponse(res);

    res = api.runLastProject();
    printResponse(res);

    for (int i = 0; i < 3; i++) {
        std::this_thread::sleep_for(std::chrono::seconds(2));
        res = api.getProjectState();
        printResponse(res);
    }

    res = api.pauseProject();
    printResponse(res);

    std::this_thread::sleep_for(std::chrono::seconds(2));
    res = api.getProjectState();
    printResponse(res);

    res = api.resumeProject();
    printResponse(res);

    std::this_thread::sleep_for(std::chrono::seconds(2));
    res = api.getProjectState();
    printResponse(res);

    res = api.stopProject();
    printResponse(res);

    std::this_thread::sleep_for(std::chrono::seconds(2));
    res = api.getProjectState();
    printResponse(res);
#endif

```

C#示例

```

res = await cobot.RunProject("Project 2024-05-28 11:36:11");
if (res.code == ResultCode.OK)
{
    Console.WriteLine("RunProject is completed");
}
else
{
    Console.WriteLine("RunProject is error: {0}", res.message);
}

```

python示例

```
res = api.runProject("test1", "test1", "test1")
printResponse(res)
```

runLastProject

接口功能

运行最后打开的工程

C++函数

```
Response runLastProject(int timeout = 30)
```

python函数

```
def runLastProject(self, timeout: int = 30) -> Response:
```

参数说明

timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "getProjectState",
  "data": {

  },
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "id": 1,
  "type": "common",
  "action": "getProjectState",
  "timestamp": 1709280418136,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0,
    "data": 0
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    // 工程运行控制接口测试
    #if 0

        res = api.runProject("test1");
        printResponse(res);

        std::this_thread::sleep_for(std::chrono::seconds(2));

        res = api.stopProject();
        printResponse(res);

        std::this_thread::sleep_for(std::chrono::seconds(2));

        res = api.runProject("test2");
        printResponse(res);

        std::this_thread::sleep_for(std::chrono::seconds(2));

        res = api.stopProject();
        printResponse(res);

        res = api.getProjectState();
        printResponse(res);

        res = api.runLastProject();
        printResponse(res);
```

```

for (int i = 0; i < 3;i++) {
    std::this_thread::sleep_for(std::chrono::seconds(2));
    res = api.getProjectState();
    printResponse(res);
}

res = api.pauseProject();
printResponse(res);

std::this_thread::sleep_for(std::chrono::seconds(2));
res = api.getProjectState();
printResponse(res);

res = api.resumeProject();
printResponse(res);

std::this_thread::sleep_for(std::chrono::seconds(2));
res = api.getProjectState();
printResponse(res);

res = api.stopProject();
printResponse(res);

std::this_thread::sleep_for(std::chrono::seconds(2));
res = api.getProjectState();
printResponse(res);
#endif

```

python示例

```

res = api.runLastProject()
printResponse(res)

```

stopProject

接口功能

停止运行工程

C++函数

```

Response stopProject(int timeout = 30)

```

C#函数

```

public async Task<Result> StopProject(int timeout = 30)

```

python函数

```
def stopProject(self, timeout: int = 30) -> Response:
```

参数说明

timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "stopProject",
  "data": {

  },
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "id": 1,
  "type": "common",
  "action": "stopProject",
  "timestamp": 1709280418136,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0,
    "data": 0
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    // 工程运行控制接口测试
    #if 0

        res = api.runProject("test1");
        printResponse(res);

        std::this_thread::sleep_for(std::chrono::seconds(2));

        res = api.stopProject();
        printResponse(res);

        std::this_thread::sleep_for(std::chrono::seconds(2));

        res = api.runProject("test2");
        printResponse(res);

        std::this_thread::sleep_for(std::chrono::seconds(2));

        res = api.stopProject();
        printResponse(res);

        res = api.getProjectState();
        printResponse(res);

        res = api.runLastProject();
        printResponse(res);

        for (int i = 0; i < 3; i++) {
            std::this_thread::sleep_for(std::chrono::seconds(2));
            res = api.getProjectState();
            printResponse(res);
        }

        res = api.pauseProject();
        printResponse(res);

        std::this_thread::sleep_for(std::chrono::seconds(2));
        res = api.getProjectState();
        printResponse(res);

        res = api.resumeProject();
        printResponse(res);
```

```
std::this_thread::sleep_for(std::chrono::seconds(2));
res = api.getProjectState();
printResponse(res);

res = api.stopProject();
printResponse(res);

std::this_thread::sleep_for(std::chrono::seconds(2));
res = api.getProjectState();
printResponse(res);
#endif
```

C#示例

python示例

```
res = api.stopProject()
printResponse(res)
```

pauseProject

接口功能

暂停工程运行

C++函数

```
Response pauseProject(int timeout = 30)
```

C#函数

```
public async Task<Result> PauseProject(int timeout = 30)
```

python函数

```
def pauseProject(self, timeout: int = 30) -> Response:
```

参数说明

timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "pauseProject",
  "data": {

  },
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "id": 1,
  "type": "common",
  "action": "pauseProject",
  "timestamp": 1709280418136,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0,
    "data": 0
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    // 工程运行控制接口测试
    #if 0
```

```
res = api.runProject("test1");
printResponse(res);

std::this_thread::sleep_for(std::chrono::seconds(2));

res = api.stopProject();
printResponse(res);

std::this_thread::sleep_for(std::chrono::seconds(2));

res = api.runProject("test2");
printResponse(res);

std::this_thread::sleep_for(std::chrono::seconds(2));

res = api.stopProject();
printResponse(res);

res = api.getProjectState();
printResponse(res);

res = api.runLastProject();
printResponse(res);

for (int i = 0; i < 3; i++) {
    std::this_thread::sleep_for(std::chrono::seconds(2));
    res = api.getProjectState();
    printResponse(res);
}

res = api.pauseProject();
printResponse(res);

std::this_thread::sleep_for(std::chrono::seconds(2));
res = api.getProjectState();
printResponse(res);

res = api.resumeProject();
printResponse(res);

std::this_thread::sleep_for(std::chrono::seconds(2));
res = api.getProjectState();
printResponse(res);

res = api.stopProject();
printResponse(res);

std::this_thread::sleep_for(std::chrono::seconds(2));
res = api.getProjectState();
printResponse(res);

#endif
```

C#示例

python示例

```
res = api.pauseProject()  
printResponse(res)
```

resumeProject

接口功能

恢复工程运行

C++函数

```
Response resumeProject(int timeout = 30)
```

C#函数

```
public async Task<Result> ResumeProject(int timeout = 30)
```

python函数

```
def resumeProject(self, timeout: int = 30) -> Response:
```

参数说明

timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "resumeProject",
  "data": {

  },
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "id": 1,
  "type": "common",
  "action": "resumeProject",
  "timestamp": 1709280418136,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0,
    "data": 0
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    // 工程运行控制接口测试
    #if 0

        res = api.runProject("test1");
        printResponse(res);

        std::this_thread::sleep_for(std::chrono::seconds(2));

        res = api.stopProject();
        printResponse(res);

        std::this_thread::sleep_for(std::chrono::seconds(2));

        res = api.runProject("test2");
```

```

    printResponse(res);

    std::this_thread::sleep_for(std::chrono::seconds(2));

    res = api.stopProject();
    printResponse(res);

    res = api.getProjectState();
    printResponse(res);

    res = api.runLastProject();
    printResponse(res);

    for (int i = 0; i < 3; i++) {
        std::this_thread::sleep_for(std::chrono::seconds(2));
        res = api.getProjectState();
        printResponse(res);
    }

    res = api.pauseProject();
    printResponse(res);

    std::this_thread::sleep_for(std::chrono::seconds(2));
    res = api.getProjectState();
    printResponse(res);

    res = api.resumeProject();
    printResponse(res);

    std::this_thread::sleep_for(std::chrono::seconds(2));
    res = api.getProjectState();
    printResponse(res);

    res = api.stopProject();
    printResponse(res);

    std::this_thread::sleep_for(std::chrono::seconds(2));
    res = api.getProjectState();
    printResponse(res);

#endif

```

python示例

```

res = api.resumeProject()
printResponse(res)

```

rs485Init

接口功能

初始化RS485，建立连接

C++函数

```
Response rs485Init(std::string type, int baudrate, int stopBit = 0, int parity = 0, int dataBit = 8, int timeout = 30)
```

C#函数

```
public async Task<Result> RS485init(string type, UInt32 baudrate, UInt32 stopBit = 0, UInt32 parity = 0, UInt32 dataBit = 8, int timeout = 30)
```

python函数

```
def rs485Init(self, type: str, baudrate: int, stopBit: int = 0, parity: int = 0, dataBit: int = 8, timeout: int = 30) -> Response:
```

参数说明

type 指定接口, "RS485": keba控制器RS485接口, "EC2RS485": 机械臂末端RS485接口
baudrate 波特率
stopBit 停止位, 0: 1停止位; 1: 1 1/2停止位; 2: 2停止位
parity 奇偶校验 0: 无; 1: 奇校验 2: 偶校验
dataBit 数据位, 注: 机械臂末端RS485接口的数据位固定为8, 不可更改
timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "rs485Init",
  "data": {
    "baudrate": baudrate,
    "stopBit": stopBit,
    "parity": parity,
    "dataBit": dataBit
  },
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "id": 1,
  "type": "common",
  "action": "rs485Init",
  "timestamp": 1709280418136,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0,
    "data": 0
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    #if 1
        // 初始化RS485
        res = api.rs485Init("EC2RS485", 115200, 0, 0, 8, 30);
        printResponse(res);

        // 写RS485
        res = api.rs485Write("EC2RS485", {0x01, 0x02, 0x03, 0x04}, 30);
        printResponse(res);

        // RS485清除读缓存
        res = api.rs485FlushReadBuffer("EC2RS485", 30);
```

```
    printResponse(res);

    // 读RS485
    res = api.rs485Read("EC2RS485", 9, 30);
    printResponse(res);

#endif
```

C#示例

```
res = await cobot.RS485init("EC2RS485", 115200);
if (res.code == ResultCode.OK)
{
    Console.WriteLine("RS485init is completed");
}
else
{
    Console.WriteLine("RS485init is error: {0}", res.message);
}
```

python示例

```
res = api.rs485Init("EC2RS485", 115200, 0, 0, 8, 30)
printResponse(res)
```

rs485Write

接口功能

通过RS485发送数据

C++函数

```
Response rs485Write(std::string type, std::vector<UINT8> msg, int timeout = 30)
```

C#函数

```
public async Task<Result> RS485write(string type, List<UInt16> msg, int timeout = 30)
```

python函数

```
res = api.rs485write(
    "EC2RS485", [0x01, 0x03, 0x00, 0x00, 0x00, 0x02, 0xc4, 0x0b], 30)
printResponse(res)
```


参数说明

type 指定接口，"RS485": keba控制器RS485接口，"EC2RS485": 机械臂末端RS485接口
msg 发送的数据，列表中按顺序表示每个字节的数值
timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "rs485write",
  "data": {
    "type": "RS485",
    "msg": "01 02 03",
  },
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "id": 1,
  "type": "common",
  "action": "rs485write",
  "timestamp": 1709280418136,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0,
    "data": 0
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"
```

```

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    #if 1
        // 初始化RS485
        res = api.rs485Init("EC2RS485", 115200, 0, 0, 8, 30);
        printResponse(res);

        // 写RS485
        res = api.rs485Write("EC2RS485", {0x01, 0x02, 0x03, 0x04}, 30);
        printResponse(res);

        // RS485清除读缓存
        res = api.rs485FlushReadBuffer("EC2RS485", 30);
        printResponse(res);

        // 读RS485
        res = api.rs485Read("EC2RS485", 9, 30);
        printResponse(res);

    #endif
}

```

C#示例

```

res = await cobot.RS485write("EC2RS485", list);
if (res.code == ResultCode.OK)
{
    Console.WriteLine("RS485write is completed");
}
else
{
    Console.WriteLine("RS485write is error: {0}", res.message);
}

```

python示例

```

res = api.rs485write(
    "EC2RS485", [0x01, 0x03, 0x00, 0x00, 0x00, 0x02, 0xc4, 0x0b], 30)
printResponse(res)

```

rs485FlushReadBuffer

接口功能

清空读缓存

C++函数

```
Response rs485FlushReadBuffer(std::string type, int timeout = 30)
```

C#函数

```
public async Task<Result> RS485flushReadbuffer(string type, int timeout = 30)
```

python函数

```
def rs485FlushReadBuffer(self, type: str, timeout: int = 30) -> Response:
```

参数说明

type 指定接口, "RS485": keba控制器RS485接口, "EC2RS485": 机械臂末端RS485接口
timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "rs485FlushReadBuffer",
  "data": {
    "type": "RS485",
  },
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "id": 1,
  "type": "common",
  "action": "rs485FlushReadBuffer",
  "timestamp": 1709280418136,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0,
    "data": 0
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;

    #if 1
        // 初始化RS485
        res = api.rs485Init("EC2RS485", 115200, 0, 0, 8, 30);
        printResponse(res);

        // 写RS485
        res = api.rs485Write("EC2RS485", {0x01, 0x02, 0x03, 0x04}, 30);
        printResponse(res);

        // RS485清除读缓存
        res = api.rs485FlushReadBuffer("EC2RS485", 30);
        printResponse(res);

        // 读RS485
        res = api.rs485Read("EC2RS485", 9, 30);
        printResponse(res);
    #endif
}
```

C#示例

```
res = await cobot.RS485flushReadbuffer("EC2RS485");
if (res.code == ResultCode.OK)
{
    Console.WriteLine("RS485flushReadbuffer is completed");
}
else
{
    Console.WriteLine("RS485flushReadbuffer is error: {0}", res.message);
}
```

python示例

```
res = api.rs485FlushReadBuffer("EC2RS485", 30)
printResponse(res)
```

rs485Read

接口功能

读取RS485收到的数据

C++函数

```
Response rs485Read(std::string type, int length, int timeout = 30)
```

C#函数

```
public async Task<Result> RS485read(string type, int length, int timeout = 30)
```

python函数

```
def rs485Read(self, type: str, length: int, timeout: int = 30) -> Response:
```

参数说明

type 指定接口, "RS485": keba控制器RS485接口, "EC2RS485": 机械臂末端RS485接口
length 读取的字节数
timeout: 发送请求超时等待时间

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

```
{
  "action": "rs485Read",
  "data": {
    "type": "RS485",
    "length": 20,
    "timeout": 5
  },
  "id": 1,
  "type": "common"
}
```

返回参数

```
{
  "id": 1,
  "type": "common",
  "action": "rs485Read",
  "timestamp": 1709280418136,
  "code": 200,
  "msg": "",
  "data": {
    "msg": "",
    "code": 0,
    "data": 0
  }
}
```

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);

    c2::CodroidApi api("192.168.101.100", "9000");
    c2::Response res;
```

```

#if 1
// 初始化RS485
res = api.rs485Init("EC2RS485", 115200, 0, 0, 8, 30);
printResponse(res);

// 写RS485
res = api.rs485Write("EC2RS485", {0x01, 0x02, 0x03, 0x04}, 30);
printResponse(res);

// RS485清除读缓存
res = api.rs485FlushReadBuffer("EC2RS485", 30);
printResponse(res);

// 读RS485
res = api.rs485Read("EC2RS485", 9, 30);
printResponse(res);

#endif

```

C#示例

```

res = await cobot.RS485read("EC2RS485", 7, 30);
if (res.code == ResultCode.OK)
{
    Console.WriteLine("RS485read is completed: List is : ");
    List<UInt16> data = (List<UInt16>)res.data!;

    foreach (UInt16 i in data)
    {
        Console.WriteLine(i);
    }
}
else
{
    Console.WriteLine("RS485read is error: {0}", res.message);
}

```

python示例

```

res = api.rs485Read("EC2RS485", 9, 30)
printResponse(res)

```

RecoveryTaskPoint

接口功能

从故障点恢复到运动恢复点

C++函数

```
Response recoveryTaskPoint(int duration = 2)
```

C#函数

```
public async Task<Result> RecoveryTaskPoint(int duration = 2)
```

python函数

```
def recoveryTaskPoint(self, duration: int = 2) -> Response:
```

参数说明

从故障点恢复到运动恢复点

:param duration 持续时间，单位秒

接口地址

192.168.101.100:9000

支持格式

json

请求方式

websocket

发送参数

返回参数

cpp示例

```
#include <iostream>
#include <vector>
#include "CodroidApi.h"

int main() {
    SetConsoleOutputCP(CP_UTF8);
```



```
c2::CodroidApi api("192.168.101.100", "9000");
c2::Response res;

#if 1
    res = api.RecoveryTaskPoint();
    printResponse(res);
#endif
```

C#示例

```
res = await cobot.RecoveryTaskPoint();
if (res.code == ResultCode.OK)
{
    Console.WriteLine("RecoveryTaskPoint is completed");
}
else
{
    Console.WriteLine("RecoveryTaskPoint is error: {0}", res.message);
}
```

python示例

```
res = api.recoveryTaskPoint()
printResponse(res)
```